Even Hand · Principles & Practices across the project lifecycle

CS5013 Programming with AI · Pre-evaluation deliverable · Team *God's Eye*: Mohamed Zayaan S (CE23B092), Pruthviraj Milind Chougale (CE23B104) · Stakeholder: Prof. Aslam Kunhi Mohamed, Dept. of Civil Engineering · `github.com/Zayaan3019/Even_Hand` · 11 October 2026

The project in four lines. Large IITM courses are marked by teams of TAs who split the scripts, so part of a relatively graded letter depends on whose pile a script landed in. Even Hand takes the instructor's marks CSV through five stages, **ingest** → **audit** → **estimate** → **outcome** → **report**, each one Maven module. It says whether the marking arrangement can be checked at all, how many marks each TA added or took away (with 95% intervals), how many letter grades those differences moved, and which ≤20 scripts deserve a second look. It runs as one Java 21 jar on the instructor's laptop: a double-click opens a three-step browser page (on 127.0.0.1) that reads the mark sheet instructors already keep, plus a CLI and a Simulation Lab. **162 tests**; a benchmark of 280 synthetic IITM courses with known answers. We used Claude Code throughout, and every practice below exists so that we can explain and defend every line it helped write.

1 Prompt engineering: prompts are engineered, versioned artefacts

Every prompt that shaped code lives in `prompts/`, one file each, with a version, what it was used for, and an **acceptance check** that must pass before its output is committed (Appendix A). The principles we apply:

Principle (course module)	How we apply it in Even Hand
Structure: role, task, context, constraints, examples, output format; tags past one screen; restate the task at the end (M2)	Every library prompt uses `<role><context><task><constraints><output>` and ends by restating the task, because attention is weakest in the middle of a long prompt.
Curated context, not the repo (M2)	We paste the failing method and what it calls, or `git diff --staged`, never whole modules. The debugging prompt for the no-effect test carried two methods and the benchmark numbers, nothing else.
Specific and negative constraints (M2)	"Java 21; no new runtime dependency"; "keep the signature"; "show only the diff"; "do not propose a patch yet"; "every number in marks, with an interval". These live at the top, where they are obeyed best.
Tests first, and the oracle is ours (M2)	Two prompts, never one: tests, then the smallest implementation. Expected values are worked by hand, e.g. totals {90,70,50×6,30,10} have mean 50 and SD 20, so the grades are S,A,B×6,C,D. The model never grades its own work.
Chain: plan → generate → critique → patch (M2)	`01-plan-a-change` (no code), `02-test-first-module`, then `05-review-a-diff` against our six invariants (no identifier on disk, no network client, every figure with an interval…), then the patch.
Iterate by naming the gap (minimal pair, M2)	Our first request for the rework was a vague paragraph ("re-design this entire project… to impress the professors"). It needed three clarifying questions back (scope, data, deadline), and the answers became lines of `01-plan-a-change` v1. v2 changes exactly one constraint, "keep the stack" → "the stack may change if every change can be explained at a whiteboard", so the difference in the plan it produced is attributable.
Data leakage is a prompt risk (M1)	No real mark or roll number ever goes into a prompt: the data-sharing note forbids it, and our examples use the synthetic `samples/` only.
Verify before you trust (M2)	Output is compiled, tested and run on a simulated course before it is committed. The benchmark then checks it against ground truth, which is how two of the defects in §2 were found.

2 Debugging: reproduce, rank, confirm, then the smallest patch

Workflow. (1) Reproduce on the smallest input: one simulated course, fixed seed. (2) Ask the assistant to *explain and rank* the likely causes, each with one experiment, and no patch yet (`03-explain-then-fix`). (3) Check the diagnosis ourselves: "a confident wrong diagnosis leads to a confident wrong patch" (M4). (4) Ask for the smallest patch, as a diff. (5) Add a regression test that fails before and passes after. (6) Commit with the *why*. When a failure is silent we make it visible first (log-driven, M4). Real cases from this project:

Symptom	Root cause, and how we found it	Fix	Guard
No re-check list on *any* realistic course	One absentee's empty exposure row differed from everyone's, so the audit called every course unmeasurable. Minimal repro: delete one student from a question-wise design.	Impute the class's TA mix only for *whole-assessment* absence; two existing tests (structural gaps) still hold	`anAbsenteeIsNotDifferentlyExposed`
No-effect test raised 4/4 false alarms on tutorial-group courses with identical TAs	Found by the benchmark's ground truth. Ranked hypotheses: the null worlds kept every student in their real pile, so the "luck of the deal" was never simulated.	Re-deal abilities in each null world (0/4 after)	`noEffectTestAllowsForTheLuckOfTheDeal`
End-to-end test: "no list" although TAs differed	Not a code bug. With 19 null simulations the smallest possible p is	39 simulations, with the reason as a comment	`PipelineTest`

	1/20 = 0.05, which is never *below* 0.05: the test's own arithmetic.		
Browser upload silently dropped ("header parser received no bytes")	The JDK server swallows handler exceptions. A catch-all that logs the trace revealed an NPE: `getOrDefault(..).isBlank() ? d : get(..)` returns null for an absent field.	`param()` helper; every request now gets an answer (400/500)	`LocalServerTest`
Page said "not responding" while the server was running	`curl -v`: empty reply. Calling the handler directly showed "*Unresolved compilation problem*": the IDE's background compiler had written a half-edited class into `target/`, and the incremental build packaged it.	`clean package`; the server now answers even on an `Error`	scan the jar for that string before a demo
A commit contained deletions its message did not mention	`git show --stat` before pushing: `git commit` takes the whole index, including deletions staged hours earlier.	`reset --mixed` to the base (unpushed), recommit path by path	stat-check before every push

3 Version control: Git for the code *and* the prompts

- **A branch per unit of work**, each opened as a pull request for review: `ci/bump-deprecated-actions` (#1), `feature/design-profiler` (#2), `review/harden-before-submission` (#3), and now `rework/simple-end-to-end`.
- **One logical change per commit, each green.** Subject in the imperative, ≤72 characters; the body says *why*. The assistant drafts *what* changed; `06-commit-message` leaves `WHY:` blank for us, because the model sees the diff, never the intent (M3). Example: "*Count an absent student's paper at the class's own mix of TAs*", whose body gives the symptom, the cause and why the fix is narrow.
- **Prompts are versioned like source.** `prompts/NN-name.md` carries a version and changelog, changes only through a commit, and names its acceptance check. A prompt change that could alter output gets its own commit (M2).
- **Generated results are versioned and reproducible.** `docs/BENCHMARK.md` and per-course CSVs come from one deterministic command; every random draw is seeded, so a diff in those files means behaviour changed.
- **Guardrails in the repo.** `.gitignore` blocks `data/`, `keys/` and every `*.csv` except `samples/`, so real marks cannot be committed by accident. `CODEOWNERS` names one owner per module. GitHub Actions runs `mvnw verify` and the audit's scenario matrix on every push.
- **Honest gap.** The history is single-author so far, which the rubric penalises. Pruthviraj owns `eh-outcome`, `eh-report` and `eh-app` and will review and commit to them under his own account before the evaluation.

4 Testing, refactoring and documentation

Layer	What it proves in Even Hand
Unit, with hand-worked oracles	Grading rules on a cohort worked on paper (mean 50, SD 20); top-10% S with ties at the cut; composites with clamping at 0 and full marks; the share of variance due to TAs = 100·25/385.94 = 6.48%.
Property-based (jqwik)	One marker of record per question exposes nobody differently, checked on 150 random allocations.
Integration & HTTP	simulate → run → reports, with no roll number in either report (roll numbers reach only the screen); the page server listens on loopback only; bad input gets a sentence saying how to fix it, not a crash.
Architecture (ArchUnit)	Only the loopback page server touches `java.net` and nothing is a network client; the estimator never depends on grading, reporting or the app.
Statistical (benchmark)	Train on 168 synthetic courses, test once on 112: TA severity within ~0.5 points where the marking links the TAs, 88–95% interval coverage, 4/24 false alarms. **80%** of flagged scripts truly moved vs **46%** for the nearest-boundary practice.
Coverage & mutation	JaCoCo: 76% of lines overall; estimator 98%, grading 94%, numerics 97%, ingest 91%, report 95%. PIT on the grading code: **80% of 168 mutants killed** (test strength 86%), after two new tests targeted the survivors.

Refactoring, mostly by subtraction. We removed Spring Boot, JPA, SQLite and the `eh-linking` module, and replaced the planned marginal-ML Rasch estimator with a 160-line backfitting class. We extracted `Pipeline.run`, which the CLI, the browser page, the lab and the benchmark all call (DRY). Refusal became a sealed type (`Emitted | Refused(reason)`) instead of exceptions. The suite stayed green across every step, and each departure from the submitted design is recorded with its reason in `docs/ARCHITECTURE.md`.

Documentation for named readers. README: three steps for a professor, three commands for a developer. `ARCHITECTURE.md`: stages, the seam, invariants each with its test, defects found. `CODE_MAP.md`: where to make a change and which test proves it. `ASSUMPTIONS.md`: A1–A16, each with its override. `BENCHMARK.md`, weekly logs, and Javadoc that explains *why*, not the signature (`07-document-for-a-reader`).

A Appendix: sample prompts from `prompts/` (abridged; full text in the repository)

01-plan-a-change.md · v2 · acceptance: one module + one test per step

```
<role>You are a senior Java engineer reviewing a student
project for IIT Madras before a live viva.</role>
<context>Even Hand audits TA marking in IITM courses: ingest
-> audit -> estimate -> outcome -> report. The approved
proposal and design must stay recognisable; the stakeholder
is a Civil Engineering professor. Modules: {{table}}. Current
state: {{what passes, what is empty}}.</context>
<task>Propose the smallest set of changes that makes the core
workflow run end to end on a realistic input, in an order
where every step leaves ./mvnw verify green.</task>
<constraints>
- Java 21, Maven, no new third-party runtime dependencies.
- Every number the instructor sees is in marks, with an
interval.
- The stack may change; prefer what can be explained at a
whiteboard.          <- the one change from v1
- Do not change the stakeholder, problem or privacy
invariants.
- If something should be cut, name the design's cut line.
</constraints>
<output>Numbered plan: step, module, files, the test that
proves it. Then three risks with fallbacks. No code yet.
</output>
Restating: a plan, not code; smallest change; green after
each step.
```

02-test-first-module.md · v1 · acceptance: module tests pass; oracle is ours

```
<task>Step 1 only: write five JUnit 5 + AssertJ tests for
GradingRule.grade(CutoffPolicy, double[]): three happy paths,
two edge cases. Do not write the implementation yet.</task>
<context>R.21.1 points S10 A9 B8 C7 D6 E4 U0 (no 5). Relative
rule: B at the mean; A +1 SD; S +2; C -1; D -2; E -3. Top 10%
(rounded up) get S; ties at the cut all get it. Worked
example, use verbatim: {90,70,50,50,50,50,50,50,30,10}
-> mean 50, population SD 20 -> S,A,B,B,B,B,B,C,D.
</context>
<constraints>Population SD (say so in a comment). No mocks,
no new dependencies. Name tests after behaviour.
</constraints>
<output>One test file. Then list assumptions you made.
</output>
(Step 2, separately: "Now the smallest GradingRule that
passes these tests. Keep the signature. Show only the
class.")
```

03-explain-then-fix.md · v1 · acceptance: regression test fails, then passes

```
Prompt 1  (diagnose, no patch)
<context>Benchmark, held-out courses: the no-effect test
rejects in 4 of 4 TUTORIAL_GROUPS courses generated with
identical TAs; expected ~5%. Correct on CE_CORE and
FIRST_YEAR_CORE (0/4 each). Code: noEffectTest, Backfit.fit
(pasted). Nothing else.</context>
<task>Explain what most plausibly produces rejections when no
effect exists. Rank the three likeliest root causes; for
each, the one experiment that confirms or eliminates it. Do
not propose a patch.</task>
Prompt 2  (after we have checked the diagnosis ourselves)
<task>Given cause #1 (null worlds keep every student in their
real pile), write the smallest patch to noEffectTest. Keep
the signature and the seeding. Show only the diff, then one
sentence on why it works.</task>
```

05-review-a-diff.md · v1 · acceptance: every BLOCKER fixed or answered

```
<task>Review the diff as the module's owner would. Findings
as BLOCKER / SHOULD / NIT, each with its line and a one-
sentence reason. Do not rewrite the code.</task>
<invariants>
1 No roll number or TA name written except in the sealed key.
2 No class uses the network beyond the loopback page server.
3 Every figure shown to an instructor carries an interval.
4 Refusal is a value with a reason, never an empty list.
5 Dependencies flow ingest->design->estimate->outcome-
>report.
6 Every random draw is seeded.
</invariants>
<diff>{{git diff --staged}}</diff>
Restating: findings only, ranked, invariants first.
```

06-commit-message.md · v1 · acceptance: imperative subject ≤72; body says why

```
<task>Draft a subject (imperative, ≤72 characters) for the
staged diff, then two or three sentences stating what
changed. Leave a line "WHY:" empty for us to fill in.</task>
<constraints>Do not guess the motivation. Do not mention
files that did not change.</constraints>
```

04-edge-cases.md · v1 · acceptance: each accepted case becomes a test or an ASSUMPTIONS line

```
<role>A test engineer who has marked exams at an Indian
university.</role>
<task>List edge cases for {{class}} that a real IITM marks
export would contain: input, expected behaviour, and whether
it is an error, a warning or normal.</task>
<constraints>Real cases first: absentees, half marks, "answer
any four of five", trailing spaces in TA names, a running-
total column mistaken for a maximum, two marks for one
answer, ties at a grade cut, a 400-student class. At most 15.
</constraints>
```

Every prompt file also records the model and settings used and a changelog; the library's conventions are in `prompts/README.md`. Architecture diagram: `docs/Even_Hand_Architecture.pdf`. Evidence for every number in this document can be regenerated with `./mvnw verify` and `java -jar eh-app/target/even-hand.jar benchmark`.